

MODULE I

BASIC PROGRAMING

1.1 PRACTICUM OBJECTIVES

The objectives for Module I Programming Basics are:

1. Students understand the flow of algorithm development, pseudocode, and flowcharts.
2. Students understand the basics of writing computer programs.
3. Students understand the structure of the C++ language.
4. Students understand and are able to use primitive data types in program development.
5. Students understand variable typecasting.

1.2 THEORETICAL BASIS

1.2.1 Algorithm

Algorithm An algorithm is a method used to solve a problem with systematic steps to obtain a better result [1]. Algorithms involve two things, namely objects and operations. An object is something that will be changed/manipulated by an algorithm, and an operation is everything that is performed on that object [2].

Consider the following example of an algorithm for making a fried egg.

- Prepare the ingredients (cooking oil, egg, salt).
- Heat the oil in a pan. Crack the egg.
- Put the egg into the pan with the hot oil.
- Add salt. Flip the egg when it is cooked.
- Remove the egg after it is perfectly cooked.
- The egg is ready to be eaten.

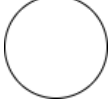
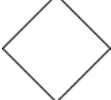
1.2.2 Pseudocode

Unlike descriptive algorithms, pseudocode algorithms are not program codes. This algorithm is an algorithmic description with a simpler structure than some programming languages to be more easily understood by humans. Pseudocode has several structures in its writing. The program declaration serves to write the name of the program that will be used. The dictionary is used to declare any variables that will be used. Furthermore, there is an algorithm description used to describe the flow of the algorithm in a program [1].

1.2.3 Flowchart

Flowchart is a chart with special symbols that aims to provide an overview related to the sequence of processes and the relationship between one process (instruction) and other processes in an algorithm. In designing a flowchart, there is actually no absolute standard that must be followed; what is important is the appropriateness of the use of symbols. Broadly speaking, there are 3 main parts in a flowchart, namely input, process, and output [3]. The flowchart symbols can be seen in **Table 1.1**.

Table 1.1 Flowchart Symbols

No.	Symbol	Name
1.		Flow Direction
2.		Terminator
3.		On Page Connector
4		Off Page Connector
5		Process
6		Decision
7		Input-Output
8		Predefine Process

Based on **Table 1.1** above, it can be known that the flow direction symbol is used to connect one symbol to another. The terminator symbol is the starting (start) and ending (stop) symbol of an activity. The on page connector symbol connects processes within the same sheet/page (labeled). It is usually used when it is difficult to connect 2 processes directly. The off page connector symbol is the same as the circle connector symbol, but this symbol is used to connect processes across different pages (labeled). The process symbol shows the processing/execution performed by the computer. The decision symbol functions for data selection that provides choices. The input-output symbol states the data input and output process. The predefine process symbol is used for the execution of a part (sub-program)/procedure [3].

1.2.4 Data Types

Data type is a type of data used to represent the kind/type of value that can be held by a variable [3]. The following are types of basic data types (primitive data types):

1. Integer

a form of data type in the form of whole numbers or numerics, not fractions. This type is usually numbers from -707.0 to 707. In this type, the stored whole numbers have a length of 16 to 65,536 bits [1].

2. Float

The data type for storing numbers that have a component in the form of a fraction. This data type is usually written in decimal form. For storage, the float data type is capable of being stored in a 32-bit area [1].

3. Boolean

The data type for choosing a correct (true) or incorrect (false) answer in the form of the numbers 1 or 0. Besides that, this data type can only describe 2 values being considered, much like a relationship of smaller or larger numbers [1].

4. Char

One of the most widely used data types besides integer. This type is used to store letters, numbers, punctuation marks, and symbols. Variables defined to hold this data type require 1 bit [1].

5. Double

The purpose of the double data type is the same as the float data type, only that double has larger storage than float. The double data type requires a storage space of 64 bits or 8 bytes [4].

1.2.5 Operators

Operators are the foundation of any programming language. As such, the functionality of the C++ programming language is incomplete or cannot run without using operators. In creating a program, there will certainly be a lot of data processed so that it produces output. The processing of that data certainly requires operators [3]. Therefore, we can define an operator as a symbol that helps us perform certain mathematical and logical calculations on operands. In other words, we can say that operators operate on operands [3]. An operator is a special symbol or character used to perform an operation helping the compiler to mathematically or logically manipulate given data. The C++ programming language provides many operators that are very useful for performing operations. There are several

types of operators in programming, namely arithmetic operators, comparison operators, logical operators, assignment operators, increment and decrement operators, and bitwise operators [3].

1.2.6 Typecasting

Typecasting is the process of converting a data type from one kind to another. In programming, typecasting is used so that a value can be processed according to the required type, for example, changing a whole number into a fraction so that division produces a decimal value [5].

1.2.7 Cin and Cout

The `cin` (console in) function is for taking an input. Its operation requires a variable to store data. `cout` is an object in C++ programming that functions to display all types of data (numeric and character) on the console screen in the form of values in variables or constants [1].

1.3 PROBLEMS

The Horizons

Rendy is trapped in a hacking simulation game called "THE HORIZONS" and must quickly penetrate three layers of enemy firewalls. Because Rendy is too nervous to calculate manually, Bina helps by creating an automated script using C++ to bypass the security. Three Layers of Security to be Breached:

1. Layer 1: Horizons Calculator

The program asks the user to enter two random numbers, then displays the precise results of addition, subtraction, multiplication, and division operations.

```

|===== THE HORIZONS =====|
|
|===== HORIZONS CALCULATOR =====|
Masukkan angka pertama : 15.5
Masukkan angka kedua   : 2
|===== HORIZONS CALCULATOR =====|
Penjumlahan : 17.5
Pengurangan : 13.5
Perkalian   : 31
Pembagian   : 7.75

```

Figure 1.1 Expected Output of Horizons Calculator

2 Layer 2: Signature Name

The program asks for a 5 letter agent code (lowercase), then the program changes the letters to capital letters, displays the ASCII value of each letter, and calculates the total and average ASCII value.

```

|===== HORIZONS SIGNATURE NAME =====|
Masukkan huruf 5 huruf (lowercase): alpro
|===== HORIZONS SIGNATURE NAME =====|
Input       : alpro
Uppercase   : ALPRO
ASCII       : 97 108 112 114 111
Total ASCII : 542
Rata-rata   : 108.4

```

Figure 1.2 Expected Output of Horizons Signature Name

3 Layer 3: Signature Name

The program asks for the range value (shift distance) of the previous letter, then the program automatically encrypts the code and reverses it using the Modulo (%) operator.

```
|===== ENKRIPSI =====|  
Masukkan range : 3  
Hasil Enkripsi : dosur  
  
|===== REVERSE =====|  
Masukkan range : 3  
Hasil Reverse : ximol
```

Figure 1.3 Expected Output of Horizons Encrypt & Reverse

1.4 RESULTS

1. Algorithm

- a. Start the program.
- b. Input two floating-point numbers: angka_pertama and angka_kedua.
- c. Calculate the sum, difference, product, and quotient of the two numbers using arithmetic operators.
- d. Display the results of the calculations.
- e. Input five lowercase characters (a, b, c, d, e).
- f. Convert each character to Uppercase by subtracting 32 from its integer value and display the result using typecasting.
- g. Calculate and display the ASCII value of each character, the total sum of the values, and the average.
- h. Input an integer range for encryption.
- i. Encrypt the characters using the formula: $((\text{char} - 'a' + \text{range}) \% 26) + 'a'$.
- j. Input an integer range for the reverse process.
- k. Calculate the reverse values and display them.
- l. End the program.

2. Pseudocode

```

Step 1:  BEGIN
Step 2:      DISPLAY "===== HORIZON CALCULATOR ====="
Step 3:      INPUT angka_pertama, angka_kedua
Step 4:      SET hasil_tambah = angka_pertama + angka_kedua
Step 5:      SET hasil_kurang = angka_pertama - angka_kedua
Step 6:      SET hasil_kali = angka_pertama * angka_kedua
Step 7:      SET hasil_bagi = angka_pertama / angka_kedua
Step 8:      DISPLAY hasil_tambah, hasil_kurang, hasil_kali, hasil_bagi
Step 9:      DISPLAY "===== HORIZON SIGNATURE NAME ====="
Step 10:     INPUT a, b, c, d, e
Step 11:     DISPLAY "UPPERCASE: ", (a-32), (b-32), (c-32), (d-32), (e-32)
Step 12:     DISPLAY "ASCII: ", INT(a), INT(b), INT(c), INT(d), INT(e)
Step 13:     DISPLAY "Total ASCII: ", SUM(ASCII values)
Step 14:     DISPLAY "Average: ", SUM(ASCII values) / 5
Step 15:     DISPLAY "===== ENKRIPSI ====="
Step 16:     INPUT range
Step 17:     SET e1 = ((a - 'a' + range) % 26) + 'a'
Step 18:     ... (repeat for e2 to e5)
Step 19:     DISPLAY e1, e2, e3, e4, e5
Step 20:     DISPLAY "===== REVERSE ====="
Step 21:     INPUT range
Step 22:     SET r1 = ((a - 'a' - range + 26) % 26) + 'a'
Step 23:     ... (repeat for r1 to r5)
Step 24:     DISPLAY r1, r2, r3, r4, r5
Step 25:  END
    
```

3. Flowchart

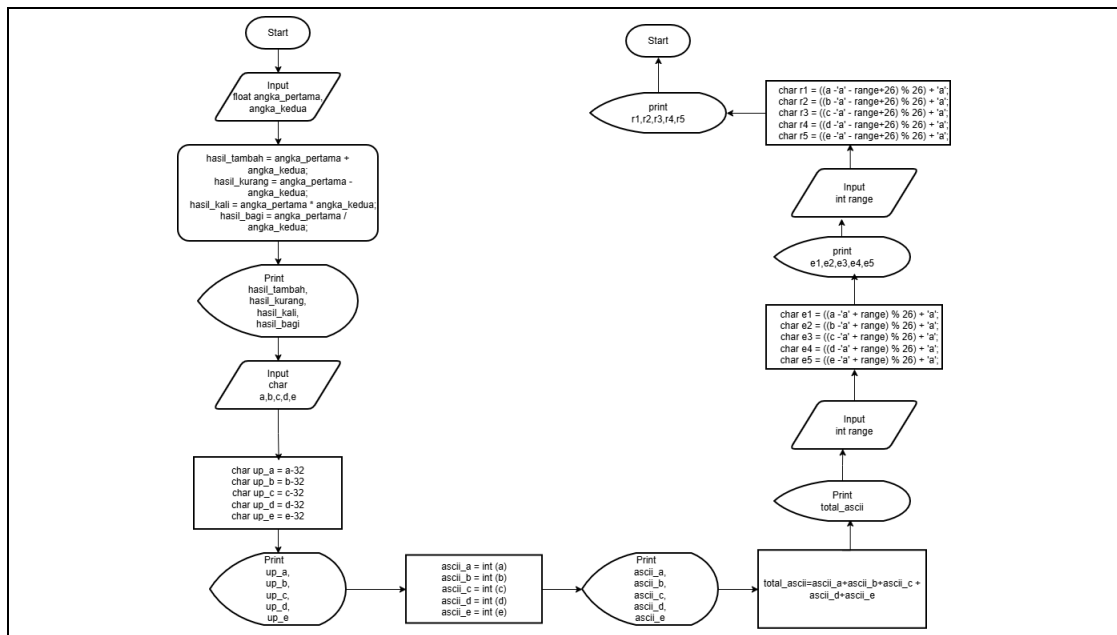


Figure 1.4 The horizons flowchart

4. Source Code

```

1. #include <iostream>
2. int main(){
3.     std::cout <<"==== HORIZON CALCULATOR =====<std::endl;
4.     float angka_pertama,angka_kedua;
5.     int range;
6.     char a,b,c,d,e;
7.     std::cout <<"masukan angka pertama: ";
8.     std::cin >> angka_pertama;
9.     std::cout <<"masukan angka kedua: ";
10.    std::cin >> angka_kedua;
11.    float hasil_tambah = angka_pertama + angka_kedua;
12.    float hasil_kurang = angka_pertama - angka_kedua;
13.    float hasil_kali = angka_pertama * angka_kedua;
14.    float hasil_bagi = angka_pertama / angka_kedua;
15.    std::cout <<"Pertambahan: "<<hasil_tambah<<std::endl;
16.    std::cout <<"Pengurangan: "<<hasil_kurang<<std::endl;
17.    std::cout <<"Perkalian: "<<hasil_kali<<std::endl;
18.    std::cout <<"Pembagian: "<<hasil_bagi<<std::endl;
19.    std::cout <<"==== HORIZON SIGNATURE NAME =====<std::endl;
20.    std::cout <<"masukan 5 karakter (lowercase): ";
21.    std::cin >> a;
22.    std::cin >> b;
23.    std::cin >> c;
24.    std::cin >> d;
25.    std::cin >> e;
26.    std::cout<<"Input: "<<a<<b<<c<<d<<e<<std::endl;
27.    std::cout<<"UPPERCASE: "<<static_cast<char>(a-
32)<<static_cast<char>(b-32)<<static_cast<char>(c-
32)<<static_cast<char>(d-32)<<static_cast<char>(e-32)<<std::endl;
28.    std::cout<<"ASCII: "<<static_cast<int>(a)<<"
"<<static_cast<int>(b)<<" "<<static_cast<int>(c)<<"
"<<static_cast<int>(d)<<" "<<static_cast<int>(e)<<" "<<std::endl;

```



```

29.     std::cout<<"Total ASCII:
      <<static_cast<int>(a)+static_cast<int>(b)+static_cast<int>(c)+static
      _cast<int>(d)+static_cast<int>(e)<<" "<<std::endl;
30.     std::cout<<"Rata-Rata:
      <<((static_cast<float>(a)+static_cast<float>(b)+static_cast<float>(c)
      +static_cast<float>(d)+static_cast<float>(e))/5<<" "<<std::endl;
31.     std::cout <<"==== ENKRIPSI =====<<std::endl;
32.     std::cout <<"Masukan Range: ";
33.     std::cin >> range;
34.     char e1 = ((a - 'a' + range) % 26) + 'a';
35.     char e2 = ((b - 'a' + range) % 26) + 'a';
36.     char e3 = ((c - 'a' + range) % 26) + 'a';
37.     char e4 = ((d - 'a' + range) % 26) + 'a';
38.     char e5 = ((e - 'a' + range) % 26) + 'a';
39.     std::cout<<"Hasil Enkripsi: "<<e1<<e2<<e3<<e4<<e5<<std::endl;
40.     std::cout <<"==== REVERSE =====<<std::endl;
41.     std::cout <<"Masukan Range: ";
42.     std::cin >> range;
43.     char r1 = ((a - 'a' - range+26) % 26) + 'a';
44.     char r2 = ((b - 'a' - range+26) % 26) + 'a';
45.     char r3 = ((c - 'a' - range+26) % 26) + 'a';
46.     char r4 = ((d - 'a' - range+26) % 26) + 'a';
47.     char r5 = ((e - 'a' - range+26) % 26) + 'a';
48.     std::cout<<"Hasil Enkripsi: "<<r1<<r2<<r3<<r4<<r5<<std::endl;
49.     return 0;
50. }

```

5. Program Result

```

===== HORIZON CALCULATOR =====
masukan angka pertama: 32
masukan angka kedua: 54
Pertambahan: 86
Pengurangan: -22
Perkalian: 1728
Pembagian: 0.592593
===== HORIZON SIGNATURE NAME =====
masukan 5 karakter (lowercase): alpro
Input: alpro
UPPERCASE: ALPRO
ASCII: 97 108 112 114 111
Total ASCII: 542
Rata-Rata: 108.4
===== ENKRIPSI =====
Masukan Range: 3
Hasil Enkripsi: dosur
===== REVERSE =====
Masukan Range: 3
Hasil Enkripsi: ximol

```

Gambar 1.5 Output of The Horizons Program

Based on **Figure 1.5**, the program uses arithmetic operators on 32 and 54 to display accurate calculation results on the console. The input "alpro" is converted to uppercase and its ASCII values are analyzed via typecasting to find a total of 542. With a range of 3, the text is encrypted to "dosur" and then reversed to "ximol" using specific modulo logic.

1.5 PROGRAM ANALYSIS

```
#include <iostream>
```

The snippet above is the header file containing standard input and output functions necessary to declare and define "cin" and "cout" statements. Without that header we can't use the "cin" and "cout" statement

```
float angka_pertama, angka_kedua;  
int range;  
char a, b, c, d, e;
```

The snippet above declaring float, int, and char represents the different data types used to determine the kind of values a variable can store. An integer is used for whole numbers, while float and double are used for numbers with decimal components

```
std::cout << "masukan angka pertama: ";  
std::cin >> angka_pertama;
```

The snippet above shows us "std" namespace is not declared or used explicitly, the programmer must add the std:: ... to standard commands.

```
std::cout << "UPPERCASE: " << static_cast<char>(a-32) << static_cast<char>(b-  
32) << static_cast<char>(c-32) << static_cast<char>(d-32) << static_cast<char>(e-  
32) << std::endl;
```

The snippet above using arithmetic symbols (+, -, *, /, %) and static_cast demonstrates the implementation of operators and typecasting. Typecasting is the process of converting one data type to another so that a value can be processed as needed, such as converting a character to an integer to obtain its ASCII value.

1.6 CONCLUSION

Based on the results gathered in Module I about Programming Basics, the following conclusions can be drawn:

1. Developing a clear understanding of algorithm flow through planning tools such as pseudocode and flowcharts is essential for ensuring a systematic and logical approach to problem-solving. These instruments allow for the structure of a program to be finalized and verified before any actual coding begins, thereby reducing logic errors.
2. Mastering the fundamental basics of computer programming is a vital skill for translating abstract logical concepts into functional, high-quality software. This foundation allows students to bridge the gap between theoretical problem-solving and the practical execution of instructions by a computer system.
3. A comprehensive knowledge of the C++ language structure, including the proper implementation of header files like `<iostream>` and strict adherence to syntax rules, is necessary for developing valid programs. Understanding these components ensures that the source code can be successfully compiled and executed within the programming environment.
4. Primitive data types—including integers, floats, characters, and booleans—serve as the indispensable building blocks for representing and managing diverse types of information within a program. Choosing the correct data type is critical for memory efficiency and ensures that the data is processed according to its specific nature, such as numerical or textual.
5. The implementation of variable typecasting provides the technical flexibility required to convert and process data between different types to meet complex logical requirements. This process is particularly useful in scenarios where a value must be temporarily treated as a different type, such as converting a character to its integer ASCII equivalent for mathematical analysis.

REFERENCE

- [1] N. Chairunnisa, *Langkah Mudah Belajar Pemrograman C++ Untuk Pemula*. Yogyakarta: Anak Hebat Indonesia, 2024.
- [2] Falahah, *Algoritma & Pemrograman: Implementasi dengan Python pada Google Colab*. Bandung: Remaja Rosdakarya, 2023.
- [3] A. B. Kaswar, Nurjannah, dan S. G. Zain, *Mudah Belajar Pemrograman Dasar C++*. Banda Aceh: Syiah Kuala University Press, 2021.
- [4] M. Nugraha, *Dasar Pemrograman dengan C++: Materi Paling Dasar Untuk Menjadi Programmer Berbagai Platform*. Yogyakarta: Deepublish, 2021.
- [5] V. Alliana and Y. Yahfizham, "Analisis Penerapan Dan Fungsi Algoritma Pemrograman Pada Komputer," *Jurnal Riset Rumpun Matematika dan Ilmu Pengetahuan Alam (JURRIMIPA)*, vol. 3, no. 1, pp. 207-218, Apr. 2024.